

NBA Történelmi adatok

Adatelemzés beadandó

Készítette: Hágen Tamás

TKR8VB

2025/26 I. félév

Tartalom

1. Elemzett adatbázis bemutatása	3
1.1. Elérhetőség, jellemzők	3
1.2. Beolvasás, tisztítás	3
2. Részfeladatok bemutatása.....	5
2.1. Egy mérkőzésen legtöbb pontot dobó játékos.....	5
2.2. Legtöbb győzelem az NBA/ABA történelmében	5
2.3. Legidősebb játékos az utolsó mérkőzésén.....	5
2.4. Legtöbb nemzetiség egy csapat egy szezonjában.....	5
2.5. A 25 legtöbb pontot átlagoló játékos átlagainak változása	6
3. Eredmények bemutatása	7
3.1. Eredmények	7
3.2. Elérhetőség	7

1. Elemzett adatbázis bemutatása

1.1. Elérhetőség, jellemzők

Az elemzett adathalmazt a kaggle.com¹ oldalon találtam, innen töltöttem le a hagyományos letöltés funkció segítségével. Folyamatosan frissül az adatbázis, a letöltés dátuma: 2025.11.26.

Az adathalmazt összesen 7 CSV file-ból áll, ezek a következők:

- PlayerStatistics.csv: Az NBA és ABA történelméből minden játékos minden meccsének a statisztikája. Minden sora egy játékos egy meccsének felel meg (~1,6M sor)
- TeamStatistics.csv: Minden mérkőzés statisztikája csapat szinten
- Games.csv: Minden mérkőzés – különbsége a TeamStatistics.csv-vel az, hogy nem kosárlabda szintű statisztikai mutatókat ad, hanem pl. nézettségszámot
- (LeagueSchedule24_25/25_26: Játékmenetrendek, nem használt)
- Players.csv: Játékos szintű adatok (születési évszám, nemzetiség stb.)
- TeamHistories.csv: Történelmi szintű csapatadatok. Egy ID több csapathoz is tartozhatott, de sosem egy időben. Azt követi végig, hogy a különböző csapatok milyen városokban játszottak, milyen utat jártak be történelmük során.

1.2. Beolvasás, tisztítás

Az adathalmaz összességében teljesnek mondható, azonban néhány lépésben kisebb utómunkát igényel. Az adatok beolvasása teljesen szokványos módon történik:

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from datetime import datetime as dt
import ownfunctions as of

games = pd.read_csv('Games.csv', low_memory = False)
players = pd.read_csv('Players.csv', low_memory = False)
playerStatistics = pd.read_csv('PlayerStatistics.csv', low_memory = False)
teamStatistics = pd.read_csv('TeamStatistics.csv', low_memory = False)
teamHistories = pd.read_csv('TeamHistories.csv', low_memory = False)
```

Ezek után felvételre kerül a playerStatistics DataFrame-be a season oszlop:

```
playerStatistics['season'] =
playerStatistics['gameDateTimeEst'].apply(of.determine_season)
```

¹ <https://www.kaggle.com/datasets/eoinamoore/historical-nba-data-and-player-box-scores>

Itt fontos megemlíteni az általam definiált függvényeket, amíg az ownfunctions.py fájlban kaptak helyet:

```
from datetime import datetime as dt

def convert_ts_to_right_format(ts):
    try:
        return dt.strptime(ts, '%Y-%m-%d %H:%M:%S').date()
    except:
        ts = ts[:6]
        return dt.strptime(ts, '%Y-%m-%d %H:%M:%S').date()

def determine_season(gameDate)-> str:
    try:
        year = gameDate.year
    except:
        gameDate = convert_ts_to_right_format(gameDate)
        year = gameDate.year
    if gameDate.month >= 10:
        return f"{year}-{year + 1}"
    else:
        return f"{year - 1}-{year}"
```

`convert_ts_to_right_format(ts)`

- A gameTimeDateEst oszlop string típusú TimeSlice-t tartalmaz. Ezt Date objektummá szeretném alakítani, hogy a year és month komponensek könnyen elérhetőek legyenek.

`determine_season(gameDate)-> str:`

- Az, hogy egy mérkőzés melyik szezonban volt játszva, attól függ, hogy az évben melyik hónapban került megrendezésre.
- Egy NBA szezon októbertől június-júliusig tart.
- Ezek alapján, ha egy mérkőzés dátumának a hónapja 10 (október), vagy a feletti érték, akkor a megfelelő szezon: year/year+1
- Ellenkező esetben: year-1/year

A későbbiekben Lambda függvényt is használok, viszont ezek a függvények hosszabbak voltak, így az átláthatóság kedvéért inkább külön helyet kaptak.

Következő adattisztítási utasítás:

```
playersWithAge = players.dropna(subset = ['birthdate'])
playersWithAge = playersWithAge[playersWithAge['birthdate'] != '1900-01-01']
```

Néhány játékosnak hiányos vagy alapérték a születési dátuma, ezt javítja a kódrészlet

Valamint vannak játékosok, akiknek az országa is üres, ezeket is tisztítja a kód:

```
teamPlayerList = teamPlayerList.merge(players[['personId', 'country']], on = 'personId', how = 'left').dropna(subset = ['country'])
```

2. Részfeladatok bemutatása

2.1. Egy mérkőzésen legtöbb pontot dobó játékos

Wilt Chamberlain 100 pontos mérkőzése a mai napig álló, talán a legrégebbi NBA rekord. Egyszerű lekérdezéssel meg kell keresni a playerStatistics DF legnagyobb pontszámát, majd ezt játékoshoz rendelni:

```
mostPoints = playerStatistics['points'].max()
mostPointsPlayer = playerStatistics[playerStatistics['points'] == mostPoints]
mostPointsName = mostPointsPlayer['firstName'] + ' ' +
mostPointsPlayer['lastName']
```

2.2. Legtöbb győzelem az NBA/ABA történelmében

A csapatok statisztikáit groupby segítségével összegzem, majd a győzelmeket (ami 1, ha győzelem, 0, ha vereség) összeadva megkapjuk az összes győzelmet. Ezek közül a legnagyobbra van szükségünk, ami a Boston Celtics lett:

```
mostWinsTeam = teamStatistics.groupby('teamId')['win'].sum().idxmax()
```

2.3. Legidősebb játékos az utolsó mérkőzésén

Összességében kicsit bonyolultabb lekérdezés, sajnos végig kell iterálni a players DF-en. Kiválasztjuk az utolsó mérkőzésüket, hozzárendeljük a karrierjük hosszát (lastGameDate – birthDate) és ezt hasonlítjuk az eddig megtalált leghosszabb karrierrel. Iteráció közben figyelni kell arra, hogy az adatbázis néhány, az NBA őskorában játszó játékosánál lehetnek hibás adatok, így ha nem találunk utolsó mérkőzést egy adott játékosnál, őt átugorjuk (természetesen ez nem is befolyásolná az eredményt).

```
for index, row in playersWithAge.iterrows():
    lastGame = playerStatistics[playerStatistics['personId'] ==
row['personId']].sort_values(by = ['gameDateTimeEst'], ascending = False)
```

Az eredmény: Bob Schaefer-nek volt összességében a leghosszabb karrierje.

2.4. Legtöbb nemzetiség egy csapat egy szezonjában

A playerStatistics DF-nek kiválasztom bizonyos oszlopait, és elhagyom a duplikáltakat a szezon alapján, így megkapva, hogy egy játékos melyik csapt(ok)nak játszott egy szezonban. Ezután ezt összekapcsolom a players DF-fel, hogy megtudjam, hogy melyik országból származnak a játékosok:

```
teamPlayerList = playerStatistics[['firstName', 'lastName', 'personId',
'playerteamCity', 'playerteamName',
'season']].drop_duplicates().sort_values(by = ['season', 'playerteamCity',
'playerteamName'], ascending = True)
teamPlayerList = teamPlayerList.merge(players[['personId', 'country']], on =
'personId', how = 'left').dropna(subset = ['country'])
```

Ezután még egyszer elvetem a duplikált sorokat. Ezzel elkészült a kombinált adathalmaz, amin alkalmazok egy groupbyt név és szezon szerint, majd az országokat megszámlolom, és a maximumukat keresem:

```
maxNationalities = teamPlayerList.groupby(['playerteamCity', 'playerteamName', 'season'])['country'].count().max()
```

Ezek után már könnyű megtalálni a csapatot, ahol a legtöbb nemzetiségű játékos játszott egy szezon alatt: Golden State Warriors, 2007-2008-as szezon.

2.5. A 25 legtöbb pontot átlagoló játékos átlagainak változása

Az összetett lekérdezésem a pontátlagok és a hozzájuk tartozó játékosok meghatározásával indul. Ezek után minden szezonban kiválasztom a top 25 játékost:

```
averages = averages.groupby('season').apply(lambda x: x.nlargest(topN, 'points')).reset_index(drop = True)
```

Ezek után meghatározom a top 25 átlagát is, majd felviszem őket a grafikonra:

```
x = np.array(averages['season'])
y = np.array(averages['points'])
z = np.array(averageTop)
v = np.array(seasons)

plt.plot(x, y, '.', color = 'green')
plt.plot(v, z, '-', color = 'red')

plt.xticks(rotation = 60)
plt.xlabel('Szezon')
plt.ylabel('Pontátlag')
```

